

Integration Manual — Grafana + ClickHouse

Grafana Cloud and On-Premise on top of an existing ClickHouse

Version: 1.2 · **Language:** English · **Verified as of:** July 2026 · **Aligned with:** repository files (Docker Compose + nginx)

Base scenario: approx. 15 users · ClickHouse already deployed at the client · Goal: fast, low-cost dashboards on top of ClickHouse.

Why the deployment matters: Grafana does not process the data; it queries it directly in ClickHouse and renders it. That is why the experience is faster the closer (network-wise) Grafana is to ClickHouse. Both deployments — on-premise and Cloud — are valid, and this manual covers both.

1. Architecture

Grafana is only the **visualization layer**. The query travels:

Browser → Grafana → (ClickHouse plugin) → ClickHouse → result → panel . The data still lives in ClickHouse; Grafana neither stores nor duplicates it.

Scenario A — On-Premise / self-hosted (recommended for this case):

```
[Users] --HTTPS--> [nginx/Caddy + SSL] --> [Grafana OSS] --Native/HTTP--> [ClickHouse]
                                     |
                                     [Grafana config database (SQLite by default)]
```

Scenario B — Grafana Cloud (SaaS):

```
[Users] --HTTPS--> [Grafana Cloud] --Native/HTTP--> [ClickHouse]
```

2. The ClickHouse connector

For this project we use the official `grafana-clickhouse-datasource` plugin, maintained and signed by Grafana Labs, open-source and **free**. It works the same way in Grafana OSS and Grafana Cloud, and it includes a query builder, prebuilt dashboards, and alerts.

Grafana Enterprise is not required either.

Note: it is best for Grafana to connect with a **read-only** ClickHouse user, restricted to the databases/tables to be visualized.

PART A — On-Premise Deployment

This manual is aligned with the repository files that accompany the installation. The reference method is **Docker Compose** (Grafana OSS + nginx as a TLS reverse proxy). Sections A3 (installation), A5 (provisioning), A6 (domain), and A7 (SSL) describe exactly what is in the repo. Installation via the APT package is kept as an alternative.

A0. Repository structure

```
.
├─ docker-compose.yaml # Grafana OSS + nginx (TLS reverse proxy)
├─ nginx.conf          # Virtual host, HTTP→HTTPS redirection and WebSocket (Grafana Live)
├─ .env.example         # Variables (domain, admin, SMTP) – copy to .env
├─ certs/              # fullchain.crt + private.key (placeholders; put the real ones)
├─ provisioning/       # Grafana configuration as code (see Section A5)
│   └─ datasources/    #   → clickhouse.yaml (ClickHouse data source, example included)
│   └─ dashboards/     #   → dashboard provider (empty)
│       └─ alerting/    #       → contact points / notifications (empty)
├─ templates/          # Alert email templates (branding) – see Section A9
└─ docs/               # This manual
```

Secrets (admin password, SMTP) and the domain are **not** hardcoded: they are read from a local `.env` file (based on `.env.example`), which is not version-controlled.

A1. Server sizing (minimum resources)

Grafana is **lightweight**: since ClickHouse does the heavy lifting, Grafana only orchestrates the query and renders. (The high requirements of 16 vCPU / 64 GB that appear in the Grafana documentation correspond to the Mimir/Loki/Tempo backends, not to Grafana as a visualization app.)

Profile	vCPU	RAM	Disk	Suggested EC2	Use
Functional minimum	1	2 GB	20 GB SSD	t3.small	Testing / POC
Recommended (15 users)	2	4–8 GB	20–30 GB SSD	t3.medium / t3.large	Production for this case
Intensive use	8+	16–32 GB	100 GB+ SSD	t3.xlarge +	Many heavy dashboards / high concurrency

- Use an SSD disk (not HDD; it impacts load time).
- Grafana does not require a large disk: it does not store the analytical data (that lives in ClickHouse), only the app and its configuration database.
- Indicative cost: a 2 vCPU / 4 GB server ≈ **USD 15–20/month** in the cloud. An existing server also works.

A2. Grafana configuration database (optional)

Grafana stores users, dashboards, and settings in an internal database. By default it uses **SQLite** (a local file), and for a **single instance with approx. 15 users this is more than sufficient and stable** — it requires nothing additional.

An external database (**PostgreSQL** or **MySQL**) is **optional** and only adds value if in the future you need **high availability** (several Grafana instances behind a load balancer) or centralized backups. For this project: **keep SQLite** and do not add complexity.

A3. Installing Grafana

Repository method — Docker Compose (`docker-compose.yaml`). It brings up two services on the same network: `grafana` (OSS) and `nginx` (TLS reverse proxy). Grafana is only published on `127.0.0.1:3000` for development; public access is via `nginx` (443). The ClickHouse plugin installs itself on startup.

Step 1 — environment variables. Copy `.env.example` to `.env` and fill in the domain and credentials:

```
cp .env.example .env
# edit .env:
#   GRAFANA_DOMAIN=grafana.tu-dominio.com
#   GF_SECURITY_ADMIN_USER=admin
#   GF_SECURITY_ADMIN_PASSWORD=CAMBIAR_ADMIN_FUERTE
```

Step 2 — `docker-compose.yaml` (excerpt of the Grafana service; the actual file also includes the `nginx` service, see Section A7):

services:

grafana:

```
image: grafana/grafana-oss:latest    # OSS (free). Pinning a version is recommended, e.g. :1:
container_name: grafana
restart: unless-stopped
```

ports:

```
- "127.0.0.1:3000:3000"                # dev only; production access is via nginx (443)
```

environment:

```
- GF_SERVER_DOMAIN=${GRAFANA_DOMAIN}
- GF_SERVER_ROOT_URL=https://${GRAFANA_DOMAIN}/
- GF_SERVER_ENFORCE_DOMAIN=true
- GF_SECURITY_ADMIN_USER=${GF_SECURITY_ADMIN_USER}
- GF_SECURITY_ADMIN_PASSWORD=${GF_SECURITY_ADMIN_PASSWORD}
- GF_USERS_ALLOW_SIGN_UP=false
- GF_USERS_DEFAULT_THEME=light
- GF_INSTALL_PLUGINS=grafana-clickhouse-datasource 3.3.0    # pinned plugin version (repro
- GF_PATHS_PROVISIONING=/etc/grafana/provisioning
```

volumes:

```
- ./templates/ng_alert_notification.html:/usr/share/grafana/public/emails/ng_alert_notific
- ./templates/ng_alert_notification.txt:/usr/share/grafana/public/emails/ng_alert_notific
- ./provisioning:/etc/grafana/provisioning    # see Section A5
- storage:/var/lib/grafana                    # Grafana config (SQLite) – back up this vol
```

volumes:

```
storage: {}
```

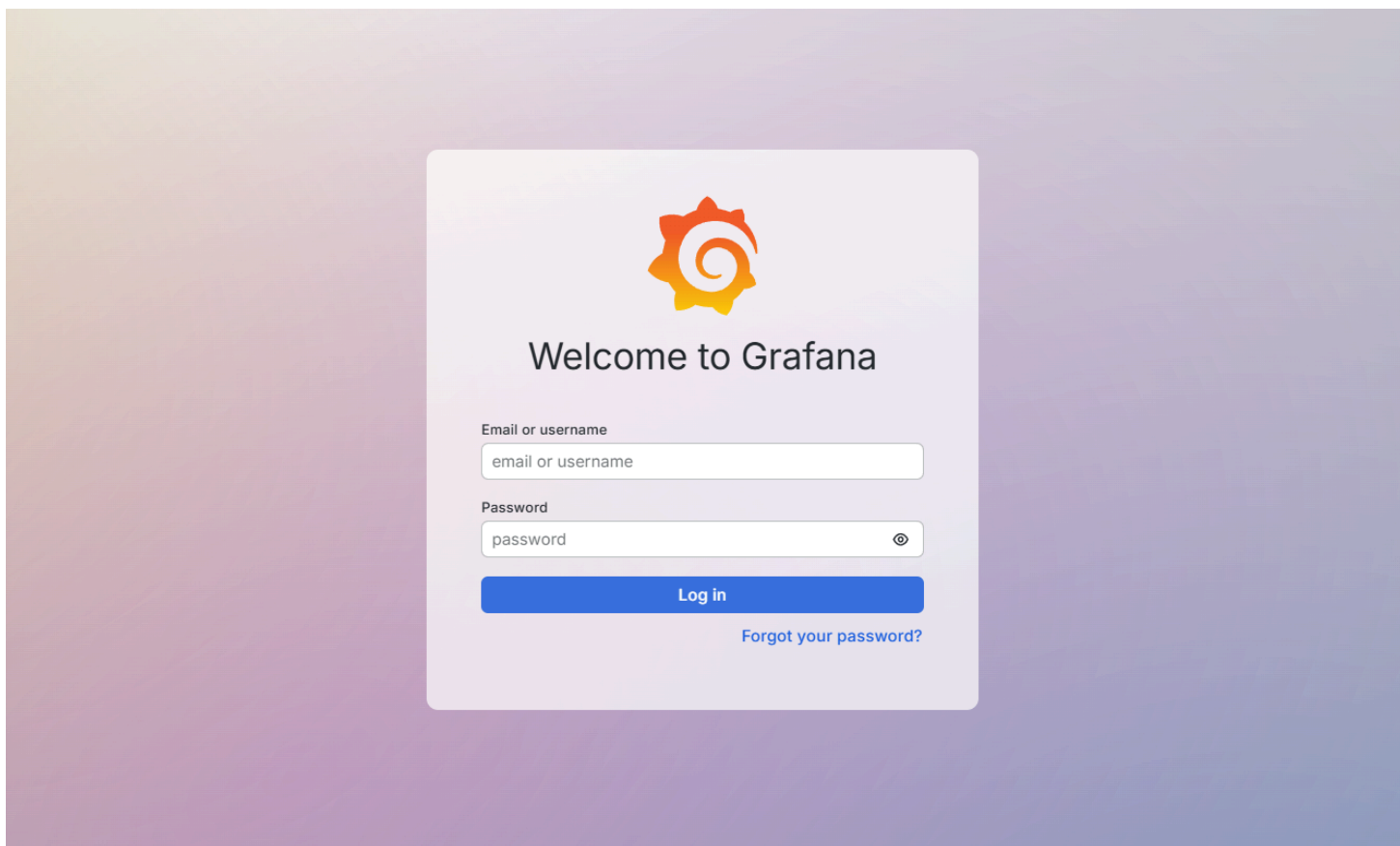
The services are brought up with `docker compose up -d`, and the logs can be followed with `docker compose logs -f grafana`.

Important configuration notes:

- **GF_SERVER_ENFORCE_DOMAIN=true requires GF_SERVER_DOMAIN / ROOT_URL to be defined** (via `.env`). If it is enabled without a domain, Grafana assumes `localhost` and breaks access through nginx.
- Configuration is managed through **GF_* environment variables**, not through `grafana.ini` (no `grafana.ini` is mounted in this template).
- **The ClickHouse plugin is installed at a pinned version** (`grafana-clickhouse-datasource 3.3.0`) to ensure reproducible deployments.

Alternative option — APT package (Ubuntu):

```
sudo apt-get install -y apt-transport-https software-properties-common
sudo mkdir -p /etc/apt/keyrings/
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo
sudo apt-get update && sudo apt-get install -y grafana # OSS
sudo grafana-cli plugins install grafana-clickhouse-datasource
sudo systemctl enable --now grafana-server
```

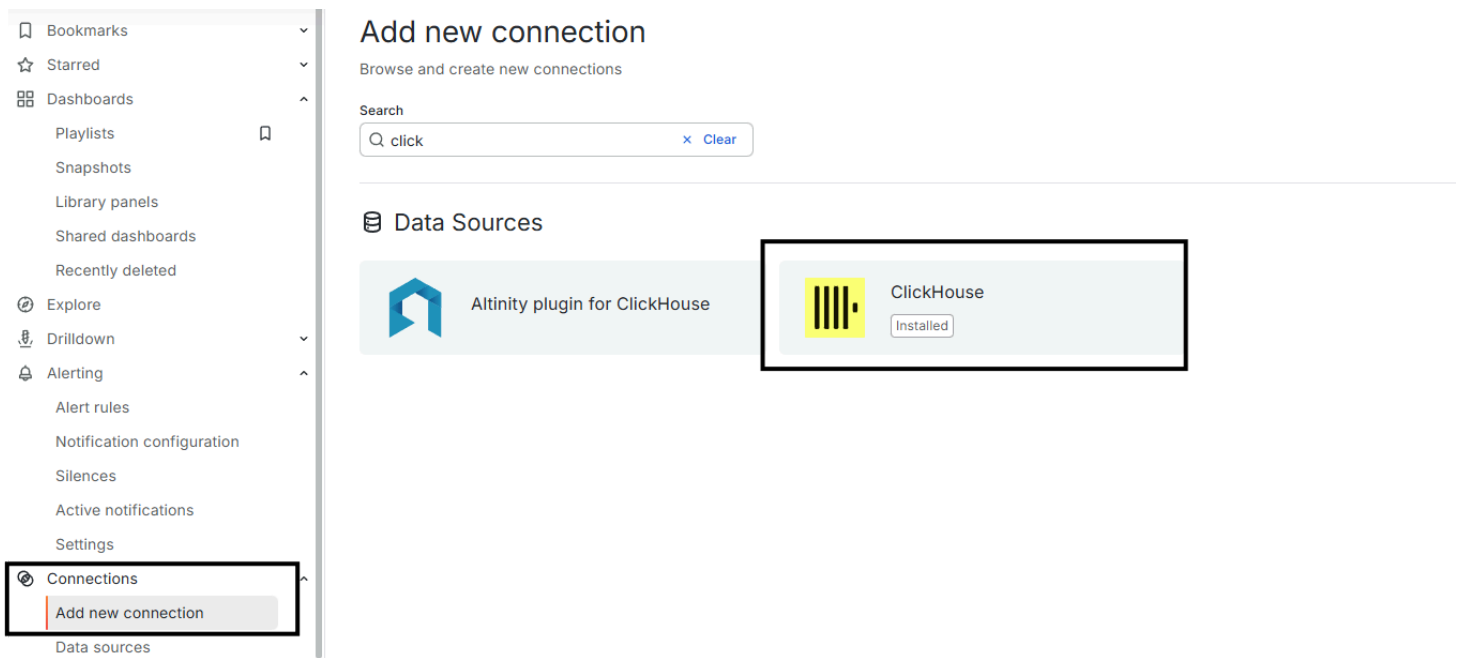


Grafana login screen already working.

A4. Install the ClickHouse plugin (if it was not done automatically)

In the UI: **Connections** → **Add new connection** → **search for "ClickHouse"** → **select the Grafana Labs plugin** → **Install**. Or via CLI:

```
grafana-cli plugins install grafana-clickhouse-datasource
# restart Grafana after installing
```



Connector catalog (*Connections* → *Add new connection*) with the official ClickHouse plugin highlighted (*Installed*).

A5. Configure the ClickHouse data source

Manual option (UI): **Connections** → **Data sources** → **Add data source** → **ClickHouse** and fill in:

Field	Value
Server host	ClickHouse host
Server port	9000 (Native) or 8123 (HTTP) — 9440 / 8443 with TLS
Protocol	Native (recommended) or HTTP
Secure / TLS	enable if ClickHouse exposes TLS
Username	grafana_ro
Password	(secret)
Default database	e.g. analitica

Click **Save & test** → the message *"Data source is working"* should appear.

Type
ClickHouse
Alerting
Supported
Explore data
Build a dashboard

Type: ClickHouse

Settings
Dashboards
Permissions
Insights
Cache

Provisioned data source
This data source was added by config and cannot be modified using the UI. Please contact your server admin to update this data source.

Name
ClickHouse
Default

Server

Server address
Server port
8123
Protocol
Native HTTP
Secure Connection

Credentials

Username
Password
configured
Reset

TLS / SSL Settings

Skip TLS Verify

Data source form (provisioned): host, port, protocol (HTTP or Native), and read-only user.

Connections
Add new connection
Data sources
Administration
General
Stats and license
Default preferences
Settings
Organizations

+ Add custom setting

✓ Data source is working
Next, you can start to visualize data by [building a dashboard from scratch](#) or by querying data in the [Explore view](#).

Delete
Test

Green "Data source is working" message after Save & test.

Option as code (provisioning — recommended for reproducibility). The repo **already includes** the ClickHouse data source in `provisioning/datasources/` , in **two versions** depending on the protocol (both with the same `name / uid` , so dashboards work with either one):

File	Protocol	Port	Status
clickhouse.yaml	HTTP	8123 (8443 TLS)	active by default
clickhouse-native.yaml.example	Native (TCP)	9000 (9440 TLS)	inactive (<code>.example</code>)

To switch from HTTP to Native: delete/rename `clickhouse.yaml` and rename `clickhouse-native.yaml.example` → `clickhouse.yaml`. Only **one** active `.yaml` should remain in the folder.

HTTP version (active):

```
apiVersion: 1
datasources:
- name: ClickHouse
  uid: clickhouse
  type: grafana-clickhouse-datasource
  access: proxy
  isDefault: true
  editable: false          # managed as code; "true" to edit in the UI
  jsonData:
    server: clickhouse.tu-host      # host or IP (field name in plugin 3.3.0)
    port: 8123                     # HTTP: 8123 · HTTPS: 8443
    protocol: http                 # native | http
    secure: false                  # true if using TLS (port 8443)
    tlsSkipVerify: false
    username: grafana_ro           # read-only user
    defaultDatabase: analitica
    timeout: "10"                  # connection timeout in seconds (string)
    queryTimeout: "60"
  secureJsonData:
    password: ${CLICKHOUSE_PASSWORD} # read from the environment variable (define in .env)
```

The Native version is identical except for `protocol: native` and `port: 9000`. The password is **not hardcoded** in either one: Grafana expands `${CLICKHOUSE_PASSWORD}` from the environment variable (defined in `.env` and passed to the container in `docker-compose.yaml`).

Field names depend on the plugin version. With the pinned version **3.3.0**, `server` and `timeout` are used. In plugin **4.x** those fields were renamed to `host` and `dialTimeout`. If you upgrade the plugin to 4.x, rename `server` → `host` and `timeout` → `dialTimeout` in the YAML files. The symptom of using the wrong name is the Grafana error **"invalid server name. Either empty or not set"**.

Structure of provisioning/ (see also `provisioning/README.md`):

- `datasources/` → `clickhouse.yaml` (**HTTP, active**) + `clickhouse-native.yaml.example` (**Native, alternative**).

- `dashboards/` → (*empty*) a YAML *provider* pointing to a folder of `.json` dashboards (see Section 3).
- `alerting/` → (*empty*) contact points and notification policies (email: see Section A9).

A6. Domain and DNS

- Register a subdomain (e.g. `grafana.tu-dominio.com`) and create a **DNS A record** pointing to the server's IP (or a `CNAME` to the load balancer). It is best for the IP to be **static**.
- Define that domain in **a single source of truth**: the `GRAFANA_DOMAIN` variable in `.env` (Grafana uses it for `GF_SERVER_DOMAIN` and `GF_SERVER_ROOT_URL`).
- **Use the same domain in `nginx.conf`** : replace `grafana.tu-dominio.com` (which appears in the `server_name` of the HTTP and HTTPS blocks) with the real value. It must match `GRAFANA_DOMAIN` .

A7. SSL certificate + reverse proxy

Repository method — nginx as a Compose service with your own certificate (bring-your-own-cert). The `docker-compose.yaml` already includes an `nginx` service that terminates TLS and proxies to `grafana:3000` over the internal network. It publishes ports 80/443 and mounts:

- `./nginx.conf` → `/etc/nginx/conf.d/default.conf`
- `./certs` → `/etc/nginx/ssl` (expects `fullchain.crt` and `private.key` ; see `certs/README.md`).

The repo's `nginx.conf` already includes: HTTP→HTTPS redirection, a parameterizable `server_name` , TLS 1.2/1.3, and the **WebSocket headers** for Grafana Live:

```

server {
    listen 80;
    server_name grafana.tu-dominio.com;
    location / { return 301 https://$host$request_uri; }
}
server {
    listen 443 ssl;
    server_name grafana.tu-dominio.com;
    ssl_certificate      /etc/nginx/ssl/fullchain.crt;
    ssl_certificate_key  /etc/nginx/ssl/private.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    location / {
        proxy_pass http://grafana:3000;
        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        # WebSocket (Grafana Live / streaming)
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}

```

You only need to place the real certificate in `certs/` and adjust the `server_name`.

Alternative 1 — Caddy (automatic HTTPS, the simplest). If you prefer not to manage certificates, replace the reverse proxy with Caddy, which issues and renews Let's Encrypt on its own:

```

# Caddyfile
grafana.tu-dominio.com {
    reverse_proxy grafana:3000
}

```

Alternative 2 — nginx + Certbot (Let's Encrypt) on the host. For installations without Docker or if you prefer the system's nginx:

```

sudo apt-get install -y nginx certbot python3-certbot-nginx
sudo certbot --nginx -d grafana.tu-dominio.com # issues TLS + automatic renewal
sudo nginx -t && sudo systemctl reload nginx

```

A8. Backups and updates

- **Backup:** with SQLite, back up the `/var/lib/grafana/grafana.db` file, which lives in this template's Docker **storage** volume. Version-controlling dashboards and data sources as code (provisioning) is the best practice.
- **Update:** `docker compose pull && docker compose up -d` (Docker) or `apt-get upgrade grafana` (package). It is worth testing first in a review environment.

A9. Email notifications (alerts)

The repo includes custom email templates in `templates/` (`ng_alert_notification.html` and `.txt`, with branding), which the `docker-compose.yaml` mounts over Grafana's default templates. For Grafana to send emails, you must configure SMTP:

1. Define the SMTP values in `.env` and uncomment the `GF_SMTP_*` lines in `docker-compose.yaml`:

```
GF_SMTP_HOST=smtp.tu-proveedor.com:587
GF_SMTP_USER=usuario_smtp
GF_SMTP_PASSWORD=secreto_smtp
GF_SMTP_FROM_ADDRESS=alertas@tu-dominio.com
GF_SMTP_FROM_NAME=Grafana
```

2. In Grafana: **Alerting** → **Contact points** → create a contact point of type *Email*.
3. Alert rules and contact points can also be version-controlled in `provisioning/alerting/`.

If email alerts are not going to be used, this section can be skipped; the mounted templates have no effect without SMTP enabled.

PART B — Grafana Cloud Deployment

B1. Create the stack

Sign up for Grafana Cloud → a stack is created with its own URL (`https://<org>.grafana.net`). Grafana Labs handles server management, TLS, and updates.

B2. Install the ClickHouse plugin in Cloud

Connections → **Add new connection** → **"ClickHouse"** → **Install** (the same free official plugin). In Cloud it is one click.

B3. Connect Grafana Cloud to ClickHouse

Grafana Cloud connects to ClickHouse using the connection details provided by the client (host, port, username, and password), just as in Section A5.

Recommendation: if the available connection is over HTTP, suggest enabling TLS/HTTPS to encrypt the traffic.

B4. Configure the data source

Same as in Section A5: fill in the host and port of the client's ClickHouse, enable **Secure/TLS** if available, and use the `grafana_ro` user.

B5. Users and roles

In Cloud, each **active viewer user** counts toward billing (see the cost analysis document, `02_Grafana_Licensing_Cost_Analysis_EN.md`). Invite only those who will actually use the dashboards.

3. Validation and performance test

1. In **Explore**, select the ClickHouse data source and run a representative business query.
2. Measure the panel's response time and verify it meets the performance goal agreed with the client.
3. Build a reference dashboard and validate filters/variables (`$__timeFilter` , ClickHouse ad-hoc filters).

4. Notes and sources

- Official ClickHouse plugin (Grafana Labs), open-source and signed:
grafana.com/grafana/plugins/grafana-clickhouse-datasource .
- Integration documentation: clickhouse.com/docs/integrations/grafana .
- Grafana OSS under the **AGPLv3** license (since April 2021); commercial and internal use permitted.

Capabilities verified in July 2026; confirm on the official pages before committing to a decision.